



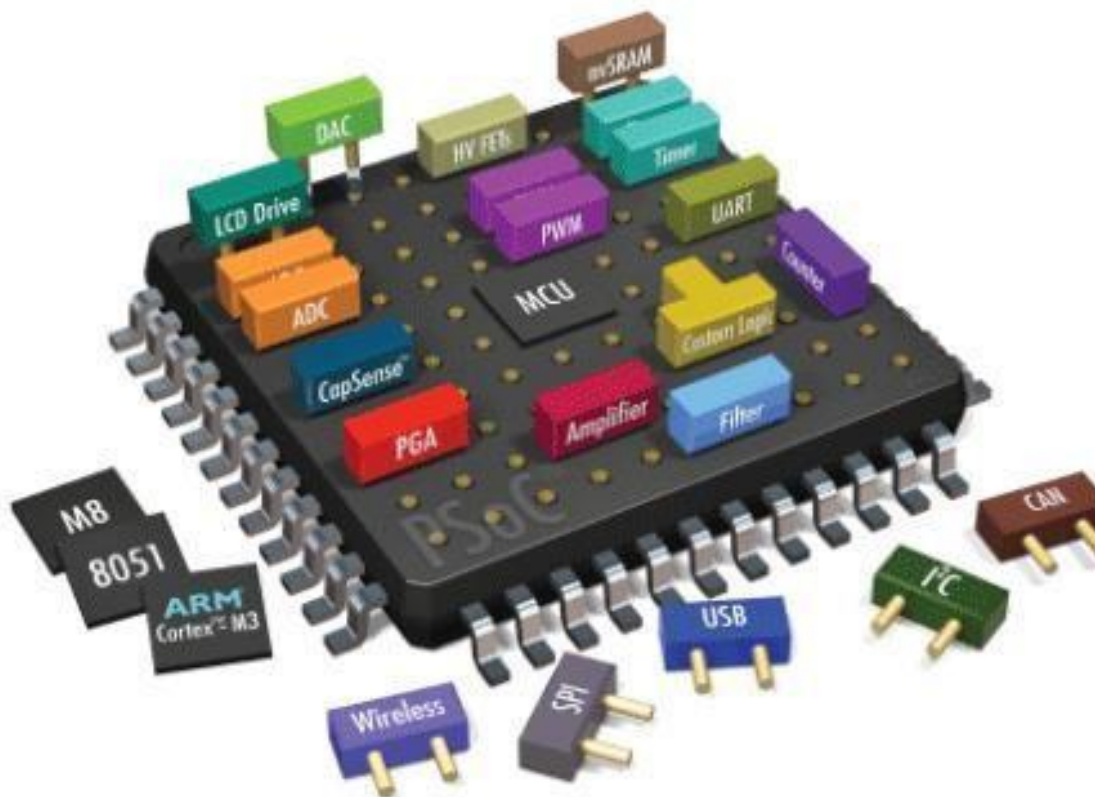
HITEC UNIVERSITY, TAXILA

**DEPARTMENT OF COMPUTER
ENGINEERING**

**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

Embedded Systems

Demonstration of Matrix Keyboard Control using 8051 Interrupts.



Dated:

Week 11

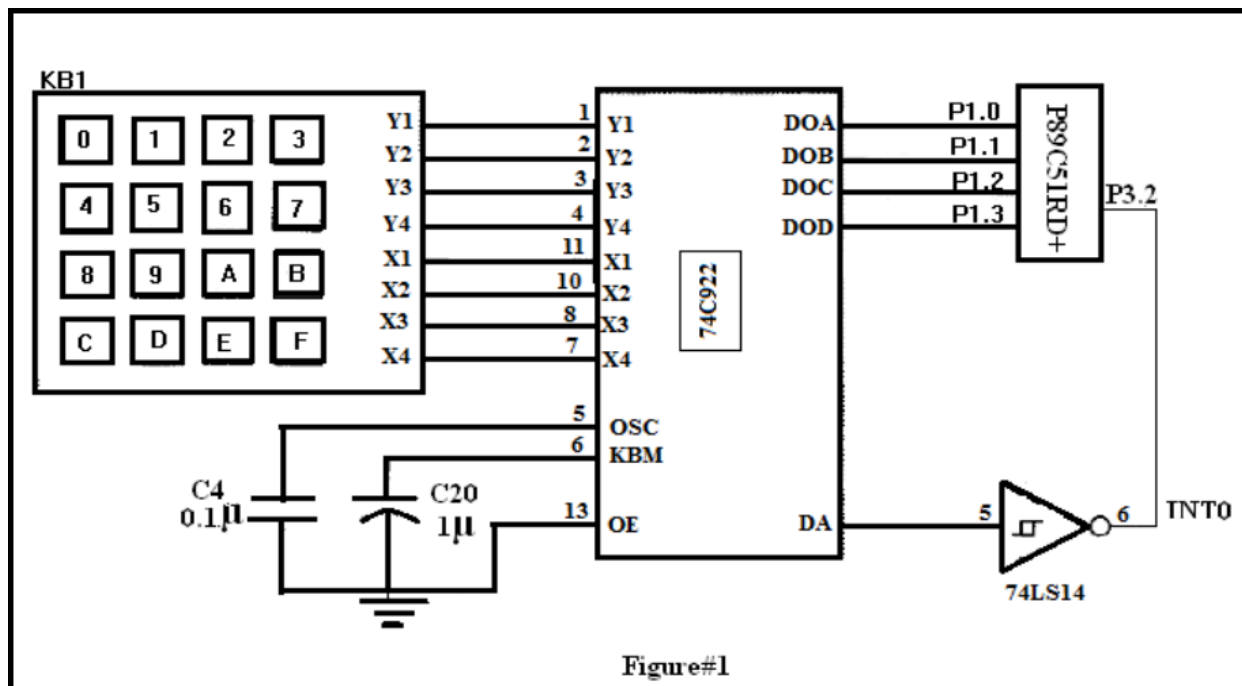
**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

Objective

1. To demonstrate the using of the keyboard circuit KB1.
2. Evaluate the operation of 74C922.

Circuit Description

The keyboard circuit is built by the matrix keyboard KB1 and encoder 74C922. Referring to figure#1 shown below, if a key is pressed, the corresponding binary value will present on the outputs DOD-DOA (P1.3-P1.0) of the encoder, as well as the data available (DA) signal on the DA output. The DA signal is inverted by an inverter and then connected to the P3.2 (INT0) pin.



Keyboard Encoder Features:

1. The output is a key value in binary.
2. It has a debouncing circuit for eliminating the noise produced by mechanical keys.
3. Tri-state output is compatible with data bus of microcomputer.
4. Pin description:
 - a. X1, X2, X3, and X4: Inputs for the column lines of the matrix keyboard.
 - b. Y1, Y2, Y3, and Y4: Inputs for the row lines of the matrix keyboard



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 12: Demonstration of Matrix Keyboard Control using 8051 Interrupts.

The relationship between X and Y is shown in Figure#2 below.

	X1	X2	X3	X4
Y1	0	1	2	3
Y2	4	5	6	7
Y3	8	9	A	B
Y4	C	D	E	F

Figure#2

The binary value on the 74C922 output is listed on Table#1.

	Key Location	Encoded Output			
		DOD	DOC	DOB	DOA
Keys in row 1	X1,Y1	0	0	0	0
	X2,Y1	0	0	0	1
	X3,Y1	0	0	1	0
	X4,Y1	0	0	1	1
Keys in row 2	X1,Y2	0	1	0	0
	X2,Y2	0	1	0	1
	X3,Y2	0	1	1	0
	X4,Y2	0	1	1	1
Keys in row 3	X1,Y3	1	0	0	0
	X2,Y3	1	0	0	1
	X3,Y3	1	0	1	0
	X4,Y3	1	0	1	1
Keys in row 4	X1,Y4	1	1	0	0
	X2,Y4	1	1	0	1
	X3,Y4	1	1	1	0
	X4,Y4	1	1	1	1

Table#1



**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

- c. DOA, DOB, DOC, and DOD: Tri-state encoded outputs. DOA is LSB and DOD is MSB.
 - d. OE: Output Enable. When this pin is low, the encoded value presents on the Encoded output pins. When this pin is high, the encoded outputs present high impedance.
 - e. DA: Data Available. When a key is pressed, a high present on the DA pin.
 - f. OSC: On-Chip oscillator input. The pin can directly receive external clock pulses or externally connects a $0.1\mu\text{F}$ capacitor.
 - g. KBM: when a $1-\mu\text{F}$ capacitor is connected to this pin, the noise caused by mechanical keys are eliminated.
- 5. CMOS configuration operates in the voltage 3-15 Volts range.
 - 6. The use of 74C922 simplifies the task of program design and improves the efficiency of execution.
 - 7. The DA signal is an active-high signal. To communicate with the microprocessor with active-low interrupt input, the DA signal should be inverted by an inverter gate.

Interrupts:

Interrupt is one of the most important and powerful concepts and features in microcontroller/processor applications. Almost all the real world and real time systems built around microcontrollers and microprocessors make use of interrupts.

What is Interrupt:

The interrupts refer to a notification, communicated to the controller, by a hardware device or software, on receipt of which controller momentarily stops and responds to the interrupt. Whenever an interrupt occurs the controller completes the execution of the current instruction and starts the execution of an Interrupt Service Routine (ISR) or Interrupt Handler. ISR is a piece of code that tells the processor or controller what to do when the interrupt occurs. After the execution of ISR, controller returns back to the instruction it has jumped from (before the interrupt was received). The interrupts can be either hardware interrupts or software interrupts.

Hardware and Software interrupt:

The interrupts in a controller can be either hardware or software. If the interrupts are generated by the controller's inbuilt devices, like timer interrupts; or by the interfaced devices, they are called



**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

the hardware interrupts. If the interrupts are generated by a piece of code, they are termed as software interrupts.

8051 Interrupts:

The 8051 controller has six hardware interrupts of which five are available to the programmer. These are as follows:

Image Showing 8051 Interrupts

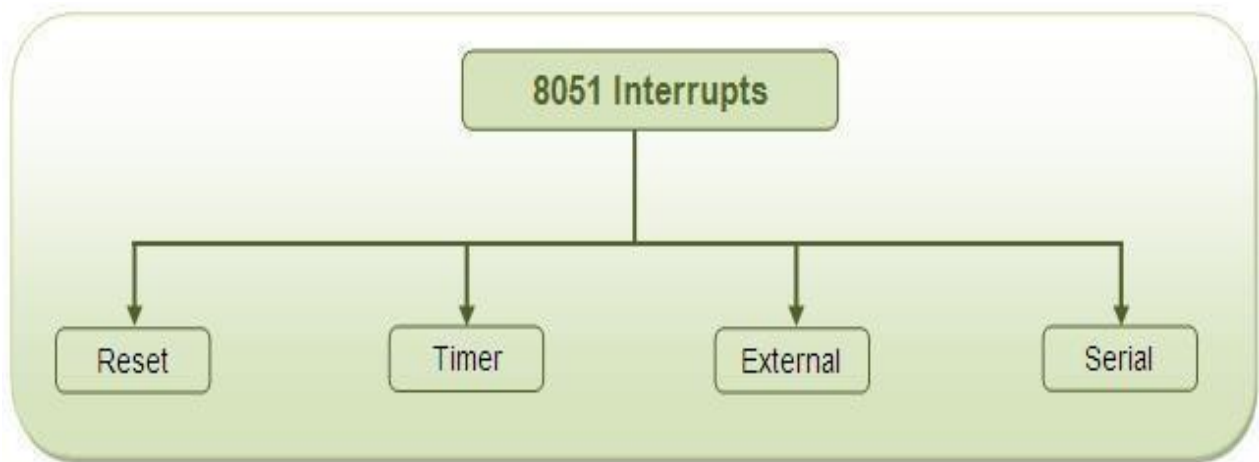


Image showing 8051 Interrupts

1. RESET interrupt – This is also known as Power on Reset (POR). When the RESET interrupt is received, the controller restarts executing code from 0000H location. This is an interrupt which is not available to or, better to say, need not be available to the programmer.
2. Timer interrupts – Each Timer is associated with a Timer interrupt. A timer interrupt notifies the microcontroller that the corresponding Timer has finished counting.
3. External interrupts – There are two external interrupts EX0 and EX1 to serve external devices. Both these interrupts are active low. In AT89C51, P3.2 (INT0) and P3.3 (INT1) pins are available for external interrupts 0 and 1 respectively. An external interrupt notifies the microcontroller that an external device needs its service.
4. Serial interrupt – This interrupt is used for serial communication. When enabled, it notifies the controller whether a byte has been received or transmitted.



**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

How is an interrupt serviced?

Every interrupt is assigned a fixed memory area inside the processor/controller. The Interrupt Vector Table (IVT) holds the starting address of the memory area assigned to it (corresponding to every interrupt).

The interrupt vector table (IVT) for AT89C51 interrupts is as follows:

Interrupt	ROM Location (Hex)	Pin	Flag clearing
Reset	0000	9	Auto
External interrupt 0	0003	12	Auto
Timer interrupt 0	000B	–	Auto
External interrupt 1	0013	13	Auto
Timer interrupt 1	001B	–	Auto
Serial COM interrupt	0023	–	Programmer clears it

When an interrupt is received, the controller stops after executing the current instruction. It transfers the content of program counter into stack. It also stores the current status of the interrupts internally but not on stack. After this, it jumps to the memory location specified by **Interrupt Vector Table (IVT)**. After that the code written on that memory area gets executed. This code is known as the Interrupt Service Routine (ISR) or interrupt handler. ISR is a code written by the programmer to handle or service the interrupt.

Programming Interrupts:

Interrupt Enable register

While programming interrupts, first thing to do is to specify the microcontroller which interrupts must be served. This is done by configuring the Interrupt Enable (IE) register which enables or disables the various available interrupts. The Interrupt Enable register has following bits to enable/disable the hardware interrupts of the 8051 controllers.

IE :	EA	-	ET2	ES	ET1	EX1	ET0	EX0
-------------	-----------	----------	------------	-----------	------------	------------	------------	------------

Bit Values of IE Register of 8051 Microcontroller

To enable any of the interrupts, first the EA bit must be set to 1. After that the bits corresponding to the desired interrupts are enabled. ET0, ET1 and ET2 bits are used to enable the Timer Interrupts



**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

0, 1 and 2, respectively. In AT89C51, there are only two timers, so ET2 is not used. EX0 and EX1 are used to enable the external interrupts 0 and 1. ES is used for serial interrupt.

EA bit acts as a lock bit. If any of the interrupt bits are enabled but EA bit is not set, the interrupt will not function. By default, all the interrupts are in disabled mode.

Note that the IE register is bit addressable and individual interrupt bits can also be accessed.

For example –

IE = 0x81; enables External Interrupt0 (EX0)

IE = 0x88; enables Serial Interrupt

Setting the bits of IE register is necessary and sufficient to enable the interrupts. **Next step** is to specify the controller what to do when an interrupt occurs. This is done by writing a subroutine or function for the interrupt. This is the ISR and gets automatically called when an interrupt occurs. It is not required to call the Interrupt Subroutine explicitly in the code.

An important thing is that the definition of a subroutine must have the keyword **interrupt** followed by the interrupt number. A subroutine for a particular interrupt is identified by this number. These subroutine numbers corresponding to different interrupts are tabulated below.

Number	Interrupt	Symbol
0	External0	EX0
1	Timer0	IT0
2	External1	EX1
3	Timer1	IT1
4	Serial	ES
5	Timer2	ET2

Interrupt priorities in 8051

The interrupts in the 8051 microcontrollers have certain priorities assigned to them. This helps the microcontroller decide which interrupt to service first if two of them occur at the same time. The priorities of the interrupts are as follows:



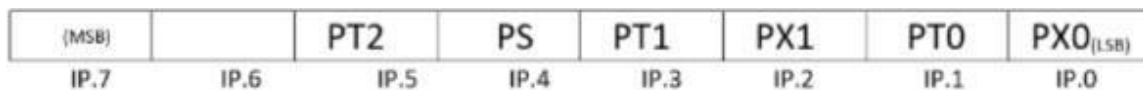
**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

The Reset interrupt has the highest priority. It is not included in this list because unlike the five interrupts mentioned in the list, the priority of the Reset interrupt cannot be altered.

1. External interrupt 0
2. Timer interrupt 0
3. External interrupt 1
4. Timer interrupt 1
5. Serial interrupt

To change the priorities of these five interrupts, the Interrupt Priority register is used. This is another register that we studied in our post on [SFRs in 8051](#). Just like the IE register, this register is bit addressable and placed at B8H in the SFR memory space.

If any bit in the IP register is set to 1, then the priority of that interrupt increases to the top (Still below Reset). However, if two bits in the IP register are raised simultaneously, then the priorities mentioned above are used to make the final decision on which interrupt will be serviced first.



INTERRUPT PRIORITY REGISTER AT B8H

If any of the bits in the IP register has a value one its priority of that interrupt is increased

- **PT2:** Increases the priority of timer 2 (for 8052 only)
- **PS:** Increases the priority of the serial interrupt
- **PT1:** Increases the priority of timer 1
- **PX1:** Increases the priority of external interrupt 1
- **PT0:** Increases the priority of timer 0
- **PX0:** Increases the priority of external interrupt 0



**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

Programming External Interrupts:

The external interrupts are the interrupts received from the (external) devices interfaced with the microcontroller. They are received at INTx pins of the controller. These can be level triggered or edge triggered. In level triggered, interrupt is enabled for a low at INTx pin; while in case of edge triggering, interrupt is enabled for a high to low transition at INTx pin. The edge or level trigger is decided by the TCON register. The TCON register has following bits:

TCON	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
------	-----	-----	-----	-----	-----	-----	-----	-----

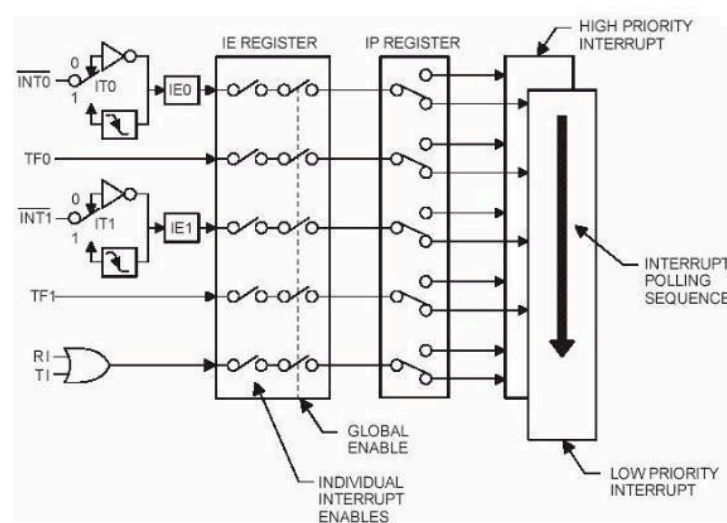
Bit Values of TCON Register of 8051 Microcontroller

Setting the IT0 and IT1 bits make the external interrupt 0 and 1 edge triggered respectively. By default, these bits are cleared, and so external interrupt is level triggered.

Note: For a level trigger interrupt, the INTx pin must remain low until the start of the ISR and should return to high before the end of ISR. If the low at INTx pin goes high before the start of ISR, interrupt will not be generated. Also, if the INTx pin remains low even after the end of ISR, the interrupt will be generated once again. This is the reason why level trigger interrupt (low) at INTx pin must be four machine cycles long and not greater than or smaller than this.

Following are the steps for using external interrupt:

1. Enable external interrupt by configuring IE register.
2. Write routine for external interrupt. The interrupt number is 0 for EX0 and 2 for EX1 respectively.





HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 12: Demonstration of Matrix Keyboard Control using 8051 Interrupts.

Interrupt System Control

1. Main program

- (1) The starting address of main program must leave from the interrupt vector area.
- (2) Specify IP to determine the priority levels of interrupt sources. If only an interrupt is to be used, this step is negligible.

(3) Specify IE to enable the interrupts to be used and disable unused interrupts.

(4) Specify IT0 or IT1 to determine the triggering type if external interrupts are used.

(5) Write main program and wait for an interrupt.

2. Interrupt service routine (ISR)

(1) Place a jump instruction in the interrupt vector and point to the starting address of interrupt service routine.

(2) Reset IE to disable other interrupts during the interrupt service routine is executing.

(3) Save the contents PSW or other registers in main program onto stack using PUSH instruction.

(4) Write interrupt service routine.

(5) Enable interrupts.

(6) Return from interrupt using RETI instruction.



**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

Exercise #1

Functional Description:

It shows the key number on the 7-seg display when a key is pressed. Place SW2_3 in ON position.

Source Program:

```
ORG    000H
JMP     START
ORG    003H
JMP     INT0

START:  ORG    100H
        MOV     SP,#30H
        MOV     P1,#0FFH
        MOV     P2,#0
        MOV     IE,#10000001B
        CLR     IT0

        JMP     $

=====
INT0:   CLR     EX0
        MOV     A,P1
        ANL     A,#0FH
        MOV     P2,A

        SETB    EX0
        RETI

        END
```

Program Description:

When a key is pressed, CPU detects the external interrupt input and executes the interrupt service routine (ISR) INT0. The ISR reads the encoded outputs of 74C922 and sends to the inputs of BCD to- 7-segment decoder.

The MOV SP, #30H instruction sets the stack pointer to memory address 30H. To wait for the interrupt, an infinite loop instruction JMP \$ is used in main program.

The CLR EX0 instruction in the INT0 ISR disables the interrupts while the service routine is running. The SETB EX0 instruction enables interrupts before from ISR. In the ISR the key value present on port 1 is read and loaded into A.

The ANL A, #0FH instruction is used to clear the higher 4 bits of A since the encoded key value exists only in the lower bits. The result is written to port 2 to show the key value on the 7-segment display.



**Lab # 12: Demonstration of Matrix Keyboard Control using
8051 Interrupts.**

Lab Tasks:

1. Perform the example in the manual and analyze the output.
2. Write a program which generate the following patterns on LED bar.
 - a) If button 0 is pressed => Turn on LED bar from left to right
 - b) If button 1 is pressed => Turn on LED bar from right to left
 - c) If button 2 is pressed => Turn on LED bar from even and odd
 - d) If button 3 is pressed => Turn on LED bar left nibble and right nibble
 - e) If button 4 is pressed => Turn on LED bar from inside to outside
 - f) If button 5 is pressed => Turn on LED bar from outside to inside

Lab Performance Evaluation:

Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Lab Report Evaluation:

Submit the Lab Report by writing all the examples and tasks which must include the following:

1. Brief description about how its work (3-5 lines)
2. The code
3. The results (Screenshot)

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports. Lab report must be submitted in group of three maximum.